

Express + TypeScript + ESM com Hot Reload

Esta apostila ensina como configurar um projeto **Express** com **TypeScript** usando **módulos ES (ESM)** e **hot reload** moderno via **tsx**.

1. Criação do Projeto

Crie uma pasta e inicialize o projeto:

```
mkdir express-ts-esm && cd express-ts-esm
npm init -y
npm i express
npm i -D typescript @types/node @types/express tsx
npx tsc --init
```

2. Configuração do package.json

Defina `"type": "module"` e adicione os scripts de desenvolvimento e produção:

```
{
  "name": "express-ts-esm",
  "version": "1.0.0",
  "type": "module",
  "private": true,
  "scripts": {
    "dev": "tsx watch --ignore dist --ignore node_modules src/server.ts",
    "build": "tsc -p tsconfig.build.json",
    "start": "node dist/server.js"
  },
  "dependencies": {
    "express": "^4.19.0"
  },
  "devDependencies": {
    "@types/express": "^4.17.21",
    "@types/node": "^22.7.0",
    "tsx": "^4.16.0",
    "typescript": "^5.6.0"
  }
}
```

```
}  
}
```

- `npm run dev` — inicia o servidor com **hot reload** (via `tsx watch`).
- `npm run build` — compila os arquivos TypeScript para JavaScript.
- `npm start` — executa o servidor compilado.

3. tsconfig.json (Configuração de Desenvolvimento)

Use o módulo **NodeNext** para habilitar importações ES modernas no Node:

```
{  
  "compilerOptions": {  
    "target": "ES2022",  
    "lib": ["ES2023"],  
    "module": "NodeNext",  
    "moduleResolution": "NodeNext",  
    "rootDir": "src",  
    "outDir": "dist",  
    "strict": true,  
    "esModuleInterop": true,  
    "skipLibCheck": true,  
    "resolveJsonModule": true,  
    "sourceMap": true,  
    "inlineSources": true,  
    "types": ["node"]  
  },  
  "include": ["src"]  
}
```

4. tsconfig.build.json (para Compilar)

```
{  
  "extends": "../tsconfig.json",  
  "compilerOptions": {  
    "noEmit": false  
  }  
}
```

5. Estrutura de Pastas

```
express-ts-esm/  
  src/  
    server.ts  
    routes/  
      hello.ts  
  package.json  
  tsconfig.json  
  tsconfig.build.json
```

6. Código do Servidor

src/server.ts

```
import express from "express";  
import { router as helloRouter } from "../routes/hello.js"; // IMPORT COM  
SUFIXO .js  
  
const app = express();  
app.use(express.json());  
  
// Rota principal  
app.use("/hello", helloRouter);  
  
const PORT = process.env.PORT ? Number(process.env.PORT) : 3000;  
app.listen(PORT, () => {  
  console.log(`[dev] Servidor rodando em http://localhost:${PORT}`);  
});
```

Atenção ao detalhe:

Como estamos usando módulos ES com "module": "NodeNext", **os imports locais precisam terminar com .js**, mesmo dentro de arquivos .ts.

Esse comportamento é necessário porque o Node espera extensões reais ao executar o JavaScript transpilado.

7. Rota Hello World

src/routes/hello.ts

```
import { Router } from "express";

export const router = Router();

router.get("/", (_req, res) => {
  res.json({ message: "Hello, World!" });
});
```

8. Executando o Servidor

Para desenvolvimento com **hot reload**:

```
npm run dev
```

A cada modificação salva em `src/`, o servidor será reiniciado automaticamente.

Para build e execução tradicional:

```
npm run build
npm start
```

9. Resultado Esperado

Acesse em seu navegador:

```
http://localhost:3000/hello
```

Resposta JSON esperada:

```
{
  "message": "Hello, World!"
}
```

Conclusão

- O **tsx watch** fornece recarga automática no desenvolvimento.
 - O **TypeScript com NodeNext** permite usar ESM moderno de forma tipada.
 - O **sufixo `.js` nos imports locais** é essencial para compatibilidade em runtime.
 - A configuração é mínima, ideal para aulas introdutórias de **Express + TypeScript moderno**.
-